```
        $.each(data.photos.photo, function(i, photo) {
          var img = $('<img>').attr('src', photo.url_q);
          $('#photos').append(img);
        });
      } else {
        alert(data.message);
      }
    },
    error: function(jqXHR, textStatus, errorThrown) {
      alert('There was an error.');
    }});
}

$(document).ready(loadPhotos);
</script>
</body>
</html>
```

error property is used instead of onerror.

The jQuery version looks similar to the original example, but the syntax varies slightly. Instead of distinct `open`, `send`, and `onload` methods, all the functionality is contained within the `ajax` method, and the HTTP request is made immediately when the `ajax` method is called.

JQuery's `dataType` property indicates that the response should be parsed as JSON. This saves the developer the additional step of using `JSON.parse` to parse the response text into a JSON object. The `XMLHttpRequest` object can do this as well by setting the `responseType` property to `'json'`. As of this writing, the `responseType` property isn't fully supported in all browsers. These are minor cosmetic differences; the functionality between the original sample and the jQuery version is the same.

While jQuery has full support for CORS, there are a few things to be aware of when using jQuery.

First, jQuery doesn't support synchronous cross-origin requests.

Second, the `ajax` method only supports `XMLHttpRequest`, and doesn't support `XDomainRequest`. This means if you're using jQuery and need support for CORS in Internet Explorer 8 or Internet Explorer 9, you'll need to write code to fall back on the `XDomainRequest` object, or use a jQuery plugin that supports `XDomainRequests`, such as the one at https://github.com/jaubourg/ajaxHooks/blob/master/src/xdr.js.

Third, if you need to set the `withCredentials` property, you'll need to use the `xhrFields` property, as shown in the following code snippet. The `xhrFields` property lets you set arbitrary fields on the `XMLHttpRequest` object:

```
$.ajax(url, {
   xhrFields: {
      withCredentials: true
   }
});
```

Fourth, jQuery doesn't set the X-Requested-With request header on cross-origin requests. JQuery traditionally sets the X-Requested-With header on HTTP requests.